



المدرسة الوطنية للعلوم التطبيقية - بني ملال
ⵜⴰⵎⴰⵔⵜ ⵜⴰⵎⴰⵏⵏⴰ ⵜⴰⵖⴻⵔⴰⵏⵜ ⵜⴰⵙⴰⵎⴰⵏⵜ
Ecole Nationale des Sciences Appliquées - Béni Mellal

Module :

Génie Logiciel

Compte Rendu du TP 3 :
Dockerisation d'une application multi-conteneurs

2^{ème} année : Filière Transformation Digitale Industrielle

Réalisé par :

ECH-CHOKHMANY Loubna

Professeur :

- Pr. BE.ELBAGHAZAOUI

Année universitaire 2025 / 2026

I. Introduction et objectifs :

Ce rapport présente la réalisation d'un travail pratique portant sur la dockerisation d'une application web multi-conteneurs. L'objectif principal est de maîtriser les concepts fondamentaux de Docker et Docker Compose, notamment la création d'images personnalisées, l'orchestration de services interdépendants et la gestion de la persistance des données.

I.1 Objectifs pédagogiques :

- Comprendre et utiliser les Dockerfile pour créer des images Docker personnalisées.
- Maîtriser Docker Compose pour orchestrer plusieurs conteneurs.
- Configurer des réseaux internes entre conteneurs.
- Mettre en place des volumes persistants pour la base de données.
- Déployer une stack complète : serveur web + base de données + interface d'administration.

I.2 Technologies utilisées :

- **Docker & Docker Compose** : plateforme de containerisation.
- **Nginx (nginx:alpine)** : Serveur web léger pour le frontend.
- **PostgreSQL 13** : Système de gestion de base de données relationnelle.
- **pgAdmin 4** : Interface web d'administration pour PostgreSQL.

II. Architecture du projet :

II.1 Structure des fichiers :

L'application est organisée selon la structure suivante :

```
project/
├── frontend/
│   ├── Dockerfile      <- Image Nginx personnalisée
│   ├── index.html      <- Page web du frontend
│   └── .dockerignore    <- Exclusion des fichiers .log
└── backend/
    └── db.env           <- Variables d'environnement PostgreSQL
```

II.2 Vue d'ensemble des services :

L'application est composée de trois services Docker interconnectés via un réseau interne commun :

Service	Image Docker	Port	Rôle
frontend	nginx:alpine	8080:80	Serveur web - affiche la page HTML
db	postgres:13	5432	Base de données PostgreSQL
pgadmin	dpape/pgadmin4	5050:80	Interface graphique de gestion BDD

II.3 Schéma d'architecture :

Les trois conteneurs communiquent entre eux via le réseau Docker interne app-network. Seuls les ports 8080 et 5050 sont exposés vers l'extérieur, tandis que PostgreSQL reste isolé sur le port 5432 accessible uniquement en interne.

III. Fichiers de Configuration :

III.1 Le fichier Dockerfile :

Le Dockerfile définit comment construire l'image Docker personnalisée pour le frontend. Il part de l'image officielle nginx:alpine (légère et sécurisée), copie la page HTML dans le répertoire par défaut de Nginx, expose le port 80 et démarre le serveur.

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

Chaque instruction joue un rôle précis :

- FROM nginx:alpine : image de base minimaliste basée sur Alpine Linux (~5MB).
- COPY : copie notre fichier HTML dans le dossier servi par Nginx.
- EXPOSE 80 : documente que le conteneur écoute sur le port 80.
- CMD : commande de démarrage de Nginx en mode foreground.

III.2 Le fichier .dockerignore :

Le fichier .dockerignore permet d'optimiser le processus de construction de l'image en excluant les fichiers inutiles du contexte de build. Cela réduit la taille du contexte envoyé au daemon Docker.

```
*.log
```

III.3 Le fichier db.env :

Ce fichier contient les variables d'environnement nécessaires pour initialiser la base de données PostgreSQL. Il est référencé dans le docker-compose.yml via la directive env_file pour des raisons de sécurité (éviter d'écrire les mots de passe en clair dans le compose).

```
POSTGRES_USER=admin
POSTGRES_PASSWORD=adminpassword
POSTGRES_DB=myapp
```

III.4 Le fichier docker-compose.yml :

C'est le fichier central du projet. Il orchestre les trois services, définit les réseaux, les volumes et les dépendances entre conteneurs.

```
version: "3.8"

services:
  frontend:
    build:
      context: ./frontend
    ports: ["8080:80"]
```

```

  networks: [app-network]

db:

  image: postgres:13

  env_file: [./backend/db.env]

  volumes: [db-data:/var/lib/postgresql/data]

  networks: [app-network]

pgadmin:

  image: dpage/pgadmin4

  environment:

    - PGADMIN_DEFAULT_EMAIL=admin@admin.com

    - PGADMIN_DEFAULT_PASSWORD=admin

  ports: ["5050:80"]

  depends_on: [db]

  networks: [app-network]

volumes:

  db-data:

networks:

  app-network:

```

III .5 Explications des directives clés :

- `build.context` : indique à Docker Compose où trouver le Dockerfile pour builder l'image.
- `ports` : mappe le port de l'hôte vers le port du conteneur (hôte:conteneur).
- `networks` : rattache le conteneur au réseau interne partagé.
- `env_file` : charge les variables d'environnement depuis un fichier externe.
- `volumes` : monte un volume nommé pour la persistance des données.
- `depends_on` : garantit que le service db démarre avant pgadmin.

IV. Mise en œuvre et résultats :

IV.1 Construction et démarrage des conteneurs :

La commande suivante construit l'image du frontend et démarre les trois services en parallèle :

```
docker-compose up --build
```

La capture d'écran ci-dessous montre le succès du lancement. On peut observer le téléchargement des images postgres:13 et dpape/pgadmin4 depuis Docker Hub, suivi de la construction de l'image personnalisée du frontend Nginx.

```
C:\Users\mon pc\Documents\ENSA_BM\CYCLE S4\Génie logiciel\Le travail à rendre\TP Docker\Code\projet>docker-compose up --build
time="2026-04-23T09:54:59+01:00" level=warning msg="C:\\Users\\mon pc\\Documents\\ENSA_BM\\CYCLE S4\\Génie logiciel\\Le travail à rendre\\TP Docker\\Code\\projet\\docker-co
mpose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 34/34
  Image postgres:13 Pulled 342.9s
  Image dpape/pgadmin4 Pulled 341.3s
#1 [internal] load local bake definitions
#1 reading from stdin 701B done
#1 DONE 0.0s

[+] up 34/35 load build definition from Dockerfile
  Image postgres:13 Pulled 342.9s
  Image dpape/pgadmin4 Pulled 341.3s
  Image projet-frontend Building 0.0s
failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory

View build details: docker-desktop://dashboard/build/default/default/xyxa778wpgae47ie3b03jyey81
```

Figure 1 : Lancement réussi de docker-compose up --build

IV.2 Démarrage et logs des conteneurs :

Une fois les trois services lancés, le terminal affiche les logs en temps réel. On peut observer les messages d'initialisation de pgAdmin 4 : configuration de l'authentification, démarrage du serveur Gunicorn sur le port 80, et activation des workers.

```
pgadmin-1 /venv/lib/python3.14/site-packages/sshtunnel.py:1040: SyntaxWarning: 'return' in a 'finally' block
pgadmin-1 return (ssh_host,
pgadmin-1 NOTE: Configuring authentication for SERVER mode.
pgadmin-1 pgAdmin 4 - Application Initialisation
pgadmin-1 =====
pgadmin-1 postfix/postlog: starting the Postfix mail system
pgadmin-1 [2026-04-23 09:10:19 +0000] [1] [INFO] Starting gunicorn 23.0.0
pgadmin-1 [2026-04-23 09:10:19 +0000] [1] [INFO] Listening at: http://[::]:80 (1)
pgadmin-1 [2026-04-23 09:10:19 +0000] [1] [INFO] Using worker: gthread
pgadmin-1 [2026-04-23 09:10:19 +0000] [123] [INFO] Booting worker with pid: 123
pgadmin-1 /venv/lib/python3.14/site-packages/sshtunnel.py:1040: SyntaxWarning: 'return' in a 'finally' block
pgadmin-1 return (ssh_host,
pgadmin-1
View in Docker Desktop View Config Enable Watch Detach
```

Figure 4 : Logs de démarrage des conteneurs - pgAdmin 4 initialise avec succès

IV.4 Vérification du frontend - <http://localhost:8080> :

La page HTML personnalisée s'affiche correctement dans le navigateur, confirmant que le conteneur Nginx fonctionne et sert bien le fichier index.html."



Bienvenue dans l'application Dockerisée

Figure 5 : Page web servie par le conteneur Nginx sur <http://localhost:8080>

IV.5 Interface pgAdmin - <http://localhost:5050>:

L'interface de connexion pgAdmin est accessible sur le port 5050. L'authentification se fait avec l'email admin@admin.com et le mot de passe admin.

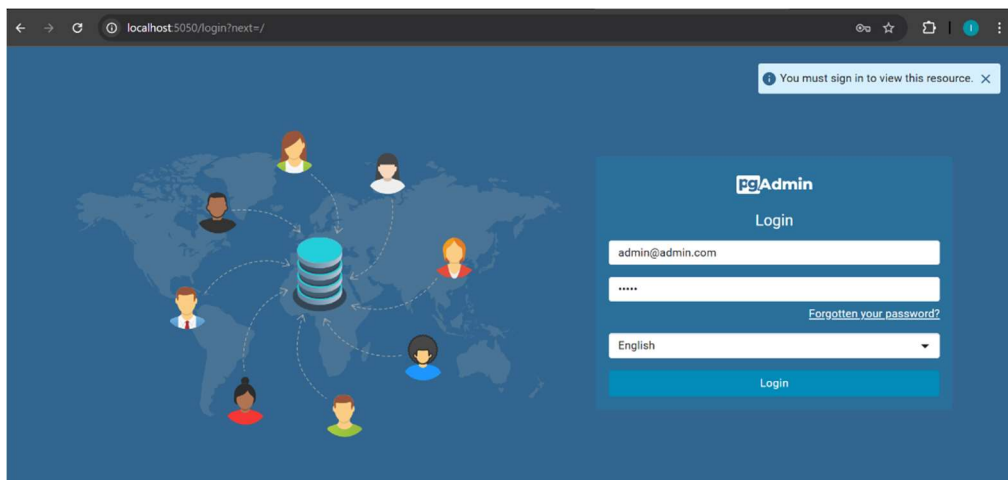


Figure 6 : Page de connexion pgAdmin 4 sur <http://localhost:5050>

Après authentification, le dashboard pgAdmin s'affiche avec les options d'administration. Le bouton Add New Server permet d'ajouter une connexion vers PostgreSQL.

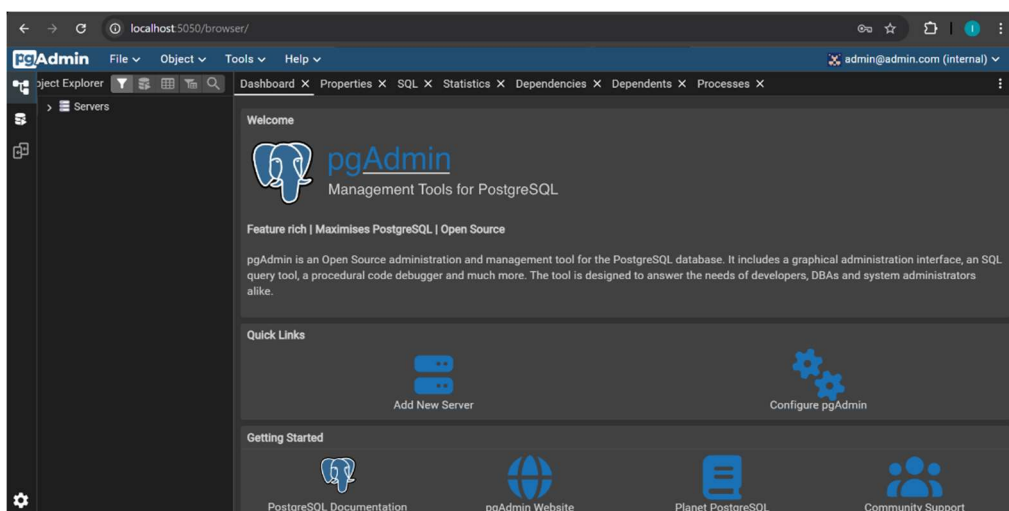


Figure 7 : Tableau de bord principal de pgAdmin 4 après connexion

IV.6 Connexion à la base de données PostgreSQL :

La fenêtre d'enregistrement du serveur. L'onglet General permet de nommer le serveur. Le nom Loubna_app est donné pour identifier la connexion.

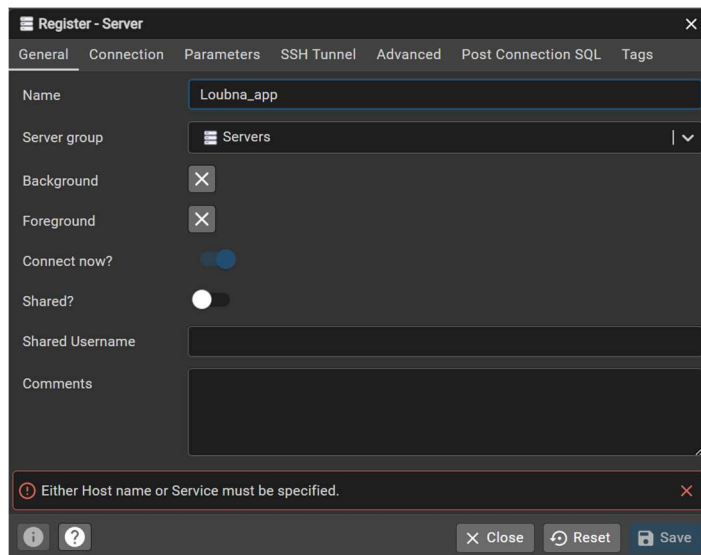


Figure 8 : Configuration du serveur - onglet General avec le nom Loubna_app

La connexion est établie avec succès. Les graphiques de monitoring s'affichent : sessions serveur, transactions par seconde, tuples in/out et Block I/O.

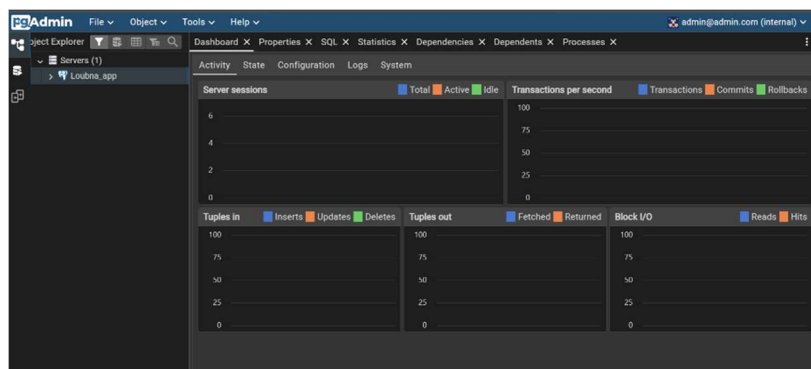


Figure 9 : Dashboard pgAdmin avec le serveur Loubna_app connecté et actif

V. Volume Persistant :

V.1 Principe et configuration :

Un volume Docker nommé db-data est configuré pour persister les données de PostgreSQL. Contrairement aux données stockées dans le conteneur (qui sont perdues à chaque suppression), le volume persiste sur le système hôte indépendamment du cycle de vie des conteneurs.

V.2 Vérification :

Le serveur Loubna_app est bien visible dans l'Object Explorer avec ses trois nœuds : Databases, Login/Group Roles et Tablespaces.

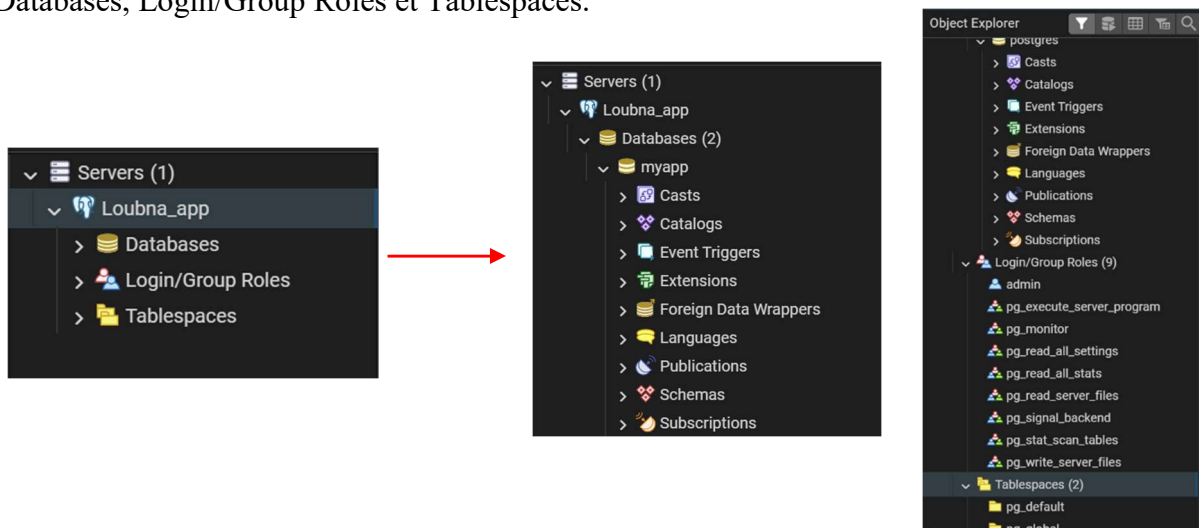


Figure 10 : Arborescence du serveur Loubna_app dans pgAdmin

6.Synthèse et Récapitulatif :

6.1 Tableau récapitulatif des étapes :

Etape	Description
Structure du projet	Creation des dossiers frontend/ backend/ et fichiers
Dockerfile	Image Nginx personnalisée pour le frontend

Etape	Description
.dockerignore	Exclusion des fichiers *.log du build
db.env	Variables d'environnement PostgreSQL
docker-compose.yml	Orchestration des 3 services
Build & Deploy	docker-compose up --build
Frontend accessible	http://localhost:8080
pgAdmin accessible	http://localhost:5050
Connexion BDD	Connexion pgAdmin vers PostgreSQL
Volume persistant	db-data pour la persistance des donnees

6.2 Points cles retenus :

- Docker Compose simplifie enormement l'orchestration multi-services avec un seul fichier de configuration.
- Les reseaux Docker permettent aux conteneurs de communiquer via leurs noms de service (ex: host: db).
- Les volumes nommes garantissent la persistance des donnees independamment du cycle de vie des conteneurs.
- Le fichier .dockerignore optimise le build en reduisant le contexte envoye au daemon Docker.
- La directive depends_on assure le bon ordre de demarrage des services.
- L'image nginx:alpine offre un serveur web leger et securise pour le frontend.

7.Conclusion :

Ce TP a permis de dockeriser une application web multi-conteneurs combinant Nginx, PostgreSQL et pgAdmin, orchestrés via Docker Compose. Les trois services communiquent sur un réseau interne dédié, les données sont persistées grâce à un volume nommé, et chaque composant remplit son rôle de manière isolée et reproductible. L'ensemble des objectifs a été atteint avec succès, confirmant l'intérêt de Docker comme outil incontournable dans le développement logiciel moderne.